

Guía Detallada del

Sistema Dinámico de Asignación de Cursos

Captura de peticiones por prioridad • Importación Excel/Link • Asignación iterativa por slot (1 curso / profesor / slot) • Bonos por desplazamiento y flexibilidad

Versión	2.1 — Agosto 2026
Stack	Next.js 16 + TypeScript + Tailwind + Prisma 7 + SQLite (better-sqlite3) + xlsx
Estado	Fase 1 (recolección) completa • Fase 2 (asignación por slot) operativa
Repositorio	asignacion-materias • dev: http://localhost:3000 • /admin

Este documento especifica el modelo de datos, flujos, reglas de validación y el algoritmo dinámico de asignación con sus cuatro factores y manejo intra-slot. Incluye ejemplos numéricos y especificación de API/UI para implementación y auditoría.

Generado: 22/08/2026 19:43 • Autores: Equipo Asignación

Contenido

1. Objetivo y alcance
2. Actores y roles
3. Requisitos funcionales
4. Modelo de datos y restricciones
5. Flujos detallados (Docente / Admin)
6. Importación Excel / Link
7. Algoritmo dinámico de asignación (4 factores, intra-slot, flex desplazado)
8. Ejemplos numéricos
9. Especificación API
10. Interfaz de usuario
11. Validaciones, casos borde y seguridad
12. Despliegue y trabajo futuro

1. Objetivo y alcance

Garantizar una distribución **equitativa, transparente y auditable** de la carga docente, respetando prioridades declaradas por cada profesor, reconociendo trayectoria y premiando flexibilidad, sin mezclar slots. El sistema opera por **slot** (solicitud): cada docente declara hasta 3 slots; cada slot contiene exactamente 3 materias alternativas con prioridad 1–3 (duplicados permitidos para empate). La asignación es secuencial: **slot 1 con materias disponibles, luego slot 2 con restantes, luego slot 3**. Dentro de cada slot un docente obtiene **máximo un curso**.

- **Fase 1:** recolección y validación de peticiones, gestión de catálogo de materias.
- **Fase 2:** cálculo de puntaje dinámico y asignación por slot con botones independientes (no ciclo automático).

Fuera de alcance Fase 2: choques de horario automáticos, balanceo de carga por horas, notificaciones.

2. Actores y roles

Actor	Identificación	Permisos
Docente	email (sin password)	Crear/editar sus peticiones por semestre (upsert por email+semester). Cargar materias, ver asignación.
Coordinador/Admin	acceso /admin (sin auth Fase 1)	Importar materias Excel/Link, ver todas peticiones, gestionar experiencia, ejecutar asignación por slot, exportar CSV, borrar asignaciones por slot.

3. Requisitos funcionales

RF-01 Peticiones

- 3 slots por docente: **slot 1 obligatorio** con 3 materias + prioridad 1–3 cada una; **slot 2 y 3 opcionales** — si se habilitan, 3 materias cada uno. Total 3–9 filas por docente/semestre.
- Prioridad requerida por materia, duplicados permitidos (**1,1,2** = empate, sin preferencia).
- code **no es único**: mismo código puede aparecer en mismo semestre con distinto horario/ubicación/días (secciones). Identidad es Course.id, no code.
- No repetir courseId dentro del mismo slot ni entre slots del mismo docente/semestre (@@unique([teacherId,semesterId,courseId])).
- Edición: Cargar mis peticiones por email+semestre y re-guardar hace upsert (transacción delete+create).

RF-02 Catálogo

- Campos mostrados al docente: code, name, cupo, dias, horario, ubicacion (cupos: Int?, dias/horario/ubicacion: String?).
- Importación Excel (.xlsx/.xls/.csv, max 5 MB, 1000 filas) o Link (SharePoint/Google Sheets/URL directa) con **confirmación obligatoria de mapeo** cada vez: headers detectados, suggestedMapping por alias, dropdown por columna code/name/cupo/dias/horario/ubicacion/ignorar, preview 5 filas.
- Modos: replace borra materias del semestre (cascade borra petitions y assignments de esos cursos) y crea; append solo añade (permite duplicados).

RF-03 Asignación

- Botones independientes por slot: **Asignar Slot 1, Asignar Slot 2, Asignar Slot 3**. Solo con materias restantes no asignadas. No ciclo automático.
- Orden obligatorio: slot 2 requiere slot 1 asignado si hay petitions en slot 1; slot 3 requiere 1 y 2.
- 1 curso / profesor / slot máximo. Curso asignado queda excluido para siguientes slots.
- Puntaje por sistema dinámico (ver §7). Desempate: puntaje desc, expYears desc, petition.id asc.

- Borrado por slot o total, y re-ejecución.

4. Modelo de datos y restricciones

Diagrama conceptual: Semester 1—* Course, Semester 1—* Petition, Teacher 1—* Petition, Course 1—* Petition, Teacher 1—* Assignment, Course 1—1 Assignment (@unique courseId), Petition 1—1 Assignment (@unique petitionId).

Tabla	Campos clave	Restricciones
semesters	id PK, label @unique (2026-2), year, term, isActive, createdAt	1 activo; cursos y asignaciones por semester
courses	id PK, code String (no único), name, cupo Int?, dias, horario, ubicacion, semesterId FK	@@index([semesterId,code]), @@index([semesterId]); code duplicable con distinto horario/ubicación
teachers	id PK, email @unique, name, expYears Int @default(0), createdAt	email lowercased; exp 0..50
petitions	id PK, teacherId, semesterId, slotNo 1..3, courseId, priority 1..3, createdAt	@@unique([teacherId,semesterId,courseId]) no repetir curso entre slots; @@index([teacherId,semesterId,slotNo])
assignments	id PK, teacherId, courseId @unique, semesterId, slotNo, petitionId @unique, priority, puntaje Int, detalle JSON, createdAt	1 curso->1 docente; 1 petition->1 assignment; @@index([semesterId,slotNo])

Tabla 1. Esquema principal (Prisma + SQLite, migrable a Postgres cambiando provider y adapter).

Nota petitions: una fila = una materia alternativa de un slot. 3 filas por slot habilitado. Ejemplo: docente con slot1 (3) + slot2 (3) = 6 filas. Prioridad por fila, no por slot.

Integridad: petitions.courseId debe pertenecer al mismo semesterId; borrado de Course con replace hace DELETE CASCADE en petitions y assignments de ese curso.

5. Flujos detallados

Docente `T`

- 1) Ingresa email+name, selecciona semestre (activo por defecto, GET /api/semesters). 2) GET /api/courses?semesterId carga catálogo con cupo/dias/horario/ubicacion. 3) Completa slot 1 (3 selects materia + 3 selects prioridad), habilita slot 2/3 si desea. 4) Valida cliente: 3 por slot habilitado, sin courseId repetido intra/inter-slot, prioridad 1..3. 5) POST /api/petitions {email,name,semesterId,slots:[{slot_no,options:[{courseId,priority}]}]} upsert transaccional. 6) Cargar mis peticiones hace GET /api/petitions?email=&semesterId; y repuebla.

Admin `admin`

- Importar materias: elige semestre destino, sube .xlsx/.xls/.csv o pega link, POST /api/admin/courses/preview (multipart o JSON url) retorna headers/preview/suggestedMapping, confirma mapeo (obligatorio code+name), elige replace/append, POST /api/admin/courses/import con mapping. Tabla materias actuales se refresca.
- Ver peticiones: GET /api/admin/petitions?semesterId agrupa por docente/slot, stats por courseId (no por code), filtros por docente y materia, export CSV.
- Experiencia: tabla docentes del semestre con input expYears 0..50, POST /api/admin/teachers/exp. Debe guardarse antes de asignar.
- Asignación: panel con 3 botones Asignar Slot 1/2/3 + Borrar Slot X + Recargar asignaciones. POST /api/admin/assignments/run {semesterId,slotNo} ejecuta motor intra-slot y persiste. GET /api/admin/assignments?semesterId lista asignaciones con detalle puntaje.
- Cada slot usa materias restantes no asignadas; docente ya asignado en ese slot no compite en siguientes prioridades del mismo slot.

6. Importación Excel / Link

Parser `src/lib/excel.ts:1 xlsx`

- Lectura: XLSX.read(buffer,{type:'buffer'}) primera hoja, sheet_to_json header:1 defval:"", filtra filas vacías, header = fila 0, rows = resto.
- Sugerencia: normaliza header toLowerCase.trim, alias code:[codigo,clave,cod] name:[nombre,materia] cupo:[capacidad] dias:[días,days] horario:[hora,schedule] ubicacion:[aula,salon,location], asigna no usado.
- Preview: 5 filas, totalRows, suggestedMapping. Confirmación obligatoria: dropdown por columna a code/name/cupo/dias/horario/ubicacion/ignorar, validar al menos code+name y sin campo duplicado.

Link

- Proxy server-side fetch(transformUrl) evita CORS. SharePoint: añade ?download=1 si falta; Google Sheets deja pasar /export?format=xlsx. Si 401/403 retorna error *link no público, descargue y suba*.
- Límites: 5 MB, 1000 filas, sanitización String(c).trim(), cupo parseado con Number(...replace(/[^0-9]/g,"")).

- Importación: valida filas code/name required, cupo int; si replace hace course.deleteMany where semesterId (cascade), luego createMany por fila; responde {imported,deleted,errors,skipped}.

7. Algoritmo dinámico de asignación

Principio: asignación por **slot secuencial** con rondas intra-slot por prioridad y puntaje dinámico. Un docente obtiene máximo un curso por slot. Cursos asignados se excluyen para slots siguientes.

4 factores (Guía §1)

Factor	Valor	Condición
Prioridad base	P1 100, P2 60, P3 30	Siempre
Bono desplazamiento	+100	P2 si perdió P1 completa; P3 si perdió P1 o P2
Flex desplazado	+30	Si prioridad previa flexible (n>1) perdida completa, suma además del bono (una sola flexible por slot, pues 3 materias)
Ajuste flexibilidad	-15 con respaldo / +30 última	Solo si total_opciones en ese nivel >1, intra-nivel
Trayectoria	+1/año max 10	teacher.expYears 0..50 truncado

Fórmula: puntaje = base(prioridad) + bonoDesplazamiento(s,flexPrev) + ajusteFlexibilidad(totalOpciones,esUltima) + puntosExperiencia(anos) src/lib/sistemaAsignacion.ts:30.

Intra-slot

- Para cada slotNo, agrupar petitions por teacher y por prioridad (1..3). totalOpciones= tamaño del grupo, esUltima= último id en el grupo.
- perdioP1 true si docente tenía P1 y ningún curso P1 le fue asignado en la ronda P1; análogo perdioP2. flexPrevPerdida true si prioridad previa tenía total>1 y se perdió completa.
- Cálculo por candidato: calcularPuntaje(Solicitud{prioridad,expYears,perdioP1,perdioP2}, Contexto{totalOpciones,esUltima}, flexPrevPerdida).
- Asignación por curso: agrupar candidatos por courseId, ordenar puntaje desc, expYears desc, petition.id asc, ganador por curso es top. Luego 1 ganador por docente (si un docente ganó 2 cursos en misma prioridad, quedarse con mejor puntaje, el otro curso queda disponible).
- Docente asignado en este slot se excluye de siguientes prioridades del mismo slot. Pasar a siguiente prioridad con perdio* actualizado.

Secuencia slots

- Ejecutar Asignar Slot 1 -> persiste Assignment con puntaje y detalle JSON. Slot 2 usa courses where id not in Assignment, igual lógica, independiente. Botones no forman ciclo, cada slot se dispara manual.

Pseudocódigo

```
for slot in [1,2,3]:
  disponibles = courses - assigned
  candidatos = petitions where slotNo==slot and course in disponibles
  perdioP1=perdioP2=flexPrev=false; asignadosSlot=set()
  for prioridad in 1..3:
    grupo = candidatos where priority==prioridad and teacher not in asignadosSlot
    calcular puntaje para cada row con totalOpciones/esUltima/flexPrev
    ganadoresPorCurso = max por courseId
    ganadoresPorTeacher = max por teacher (1 curso)
    persistir ganadores, marcar asignadosSlot
    actualizar perdioP*/flexPrev para siguiente prioridad
```

8. Ejemplos numéricos

Caso flexible con desplazamiento (intra-slot 2, exp 5): docente tiene slot con P1 n=2 (flex) y P2 n=1

Opción	Cálculo	Total
P1 opción1 (no última)	100 +0 + (-15) +5	90
P1 opción2 (última)	100 +0 +30 +5	135
Si pierde ambas P1 -> P2	60 +100 +30(flex desplazado) +0 +5	195

Con 3 materias por slot solo una prioridad puede ser flexible (2+1 o 3). Rígido n=1 flexibilidad 0. Desempate por expYears y petition.id.

Validación real slot 2: Ana P1 2 opciones -> 138 gana 144, Bob P1 2 opciones -> 132 gana 145, David perdió P1 flexible 144/145 y ganó P2 147 con 191 (60+130+0+1).

9. Especificación API

Método	Ruta	Body / Query	Descripción
GET	/api/semesters	-	Lista semestres, isActive
GET	/api/courses?semesterId	semesterId	Catálogo con cupo/días/horario/ubicacion
GET	/api/petitions?email=&semesterId	email, semesterId	Peticiones del docente agrupadas por slot
POST	/api/petitions	{email,name,semesterId,slots:[{slot_no,options:[{courseId,priority}]}]}	Upsert transaccional, valida 3 por slot, sin duplicar courseId
GET	/api/admin/petitions?semesterId	semesterId	Todas peticiones agrupadas + statsByCourse
POST	/api/admin/courses/preview	multipart file o JSON url	Parsea Excel, retorna headers, preview 5 filas, suggestedMapping
POST	/api/admin/courses/import	file/url + mapping + semesterId + mode	Valida mapping code+name, crea cursos, replace/append
GET	/api/admin/teachers/exp?semesterId	semesterId	Lista docentes con expYears
POST	/api/admin/teachers/exp	{email,expYears}	Actualiza trayectoria
GET	/api/admin/assignments?semesterId	semesterId	Lista asignaciones con detalle
POST	/api/admin/assignments/run	{semesterId,slotNo}	Ejecuta motor por slot, persiste
DELETE	/api/admin/assignments?semesterId&slotNo;	query	Borra asignaciones por slot o todas

10. Interfaz de usuario

Docente `I`

- Email + Nombre + Semestre + Cargar mis peticiones
- Slot 1 obligatorio 3 materias (select con code - name | Cupo:X | días horario | ubicacion) + prioridad 1..3 cada una; Slots 2/3 checkbox habilitado + 3 selects c/u; validación cliente duplica; Guardar peticiones POST; mensaje ok/error.

Admin `/admin`

- Importar Materias: semestre destino, file y/o link, Previsualizar, tabla mapeo por columna (dropdown obligatorio), modo replace/append, Confirmar e Importar, tabla materias actuales.
- Experiencia: tabla docentes del semestre con input expYears y Guardar.
- Asignación por Slot: 3 botones Asignar Slot 1/2/3, Borrar Slot X y tabla asignaciones con slot,docente,curso,prioridad,puntaje,detalle base+desp+flex+exp.
- Peticiones: filtros por docente y materia, stats demanda por materia, export CSV con cupo/días/horario/ubicacion.

11. Validaciones, casos borde y seguridad

- Petitions: slot 1 requiere 3; slots 2/3 si habilitado requieren 3; prioridad 1..3 required; sin courseId repetido intra/inter-slot; course.semesterId debe coincidir.
- Cursos: code no único; cupo int >=0; replace borra cursos del semestre y cascade petitions/assignments; append permite duplicados exactos.
- Asignación: curso ya asignado excluido; docente ya asignado en slot excluido; petición sin curso disponible se reporta en noAsignados con razón. DELETE permite re-ejecutar.
- Excel: 5 MB, header requerido, code+name mapeados obligatorio, sin campo duplicado, filas sin code/name van a errors. Link SharePoint requiere público o ?download=1.
- Seguridad Fase 1: sin auth, identificación por email; admin sin protección (añadir middleware Fase 3). Rate limit recomendado en import.

12. Despliegue y trabajo futuro

- Instalación: npm install; npx prisma generate; npx prisma migrate dev; npx tsx prisma/seed.ts; npm run dev (3000) y npm run build && npm start. SQLite dev.db migrable a Postgres cambiando provider y @prisma/adapters-pg en src/lib/prisma.ts:1.
- Futuro: choque de horarios automático, balanceo por horas/cupo, notificaciones, historial semestral, roles con NextAuth, tests e2e, auditoría de puntaje por curso.

Anexo: archivos clave prisma/schema.prisma:1, src/lib/sistemaAsignacion.ts:1, src/lib/asignador.ts:1, src/app/api/admin/assignments/run/route.ts:1, sistema_asignacion.py:1 (referencia Python independiente).

Fin del documento.